

WORLD ROBOT OLYMPIAD 2026
FUTURE ENGINEERS · SELF-DRIVING CARS

ROBOVANGUARD

Engineering Documentation

Ramco Institute of Technology

Rajapalayam, Tamil Nadu, India

Abishek Kumar V · Mirjone D · Manojkumar M

Mentor: Dr. S. Valai Ganesh

github.com/mabamano/WRO_2026-RoboVanguard

Revision 2.0 · National Finals submission

Contents

Page numbers update automatically. In Word, select the table and press F9, or right-click it and choose "Update Field" → "Update entire table".

Overview	4
1. Team	4
2. Design Brief and Constraints.....	4
2.1 Hard constraints from the rules.....	4
2.2 Self-imposed constraints.....	5
2.3 Vehicle specification	5
3. System Architecture	6
3.1 How the subsystems constrain each other	6
4. Mobility and Mechanical Design	8
4.1 Why steered front wheels and not a differential drive	8
4.2 Steering geometry, and what we actually measured	8
4.3 Turning radius versus lane width.....	9
4.4 Drive train: torque, speed and acceleration.....	10
4.5 Chassis, materials and centre of gravity.....	10
4.6 Mechanical iteration log	11
5. Power and Sensor Architecture	13
5.1 The RoboGuard carrier board	13
5.2 Power budget	13
5.3 Sensing strategy: vision-primary, ultrasonic as a gate.....	15
5.4 Camera placement, justified by field geometry	16
5.5 Wiring and pin map.....	16
5.6 Colour calibration procedure	17
6. Software Architecture and Obstacle Strategy	19
6.1 Module map	19
6.2 Why contour area is the control signal.....	19
6.3 Pi ↔ ESP32 serial contract.....	20
6.4 Round 1 — Open Challenge	21
6.5 Round 2 — Obstacle Challenge.....	22
6.6 Parking.....	23
7. Engineering Decisions and Trade-offs	25
8. Testing, Tuning and Validation	27
8.1 Test methodology	27
8.2 Gain tuning	27
8.3 Results	27
8.4 Test log	28
8.5 System-level measurements.....	28
9. Risk Register and Failure Modes.....	29
10. Build and Reproduce	31
10.1 Bill of materials	31
10.2 Mechanical assembly	31

10.3 ESP32 firmware	31
10.4 Raspberry Pi software	32
10.5 Running.....	32
10.6 Selecting and starting a round	32
11. Known Limitations and Roadmap.....	33
12. Repository Structure and Versioning.....	33
12.1 Development trajectory.....	33
12.2 Demonstration videos.....	34
Appendix A — Vehicle Photographs.....	35
Appendix B — Team.....	38

Overview

An autonomous 1:14-scale vehicle with Ackermann-type steering, built for the WRO 2026 Future Engineers Self-Driving Cars challenge. The vehicle completes the Open Challenge (three laps with a randomised interior wall) and the Obstacle Challenge (three laps past red and green traffic signs, followed by parking) using a forward-facing wide-angle camera as its primary perception sensor, a Raspberry Pi 5 for vision, and a custom ESP32 carrier board for real-time actuation.

This document is the primary engineering record for the vehicle. It is organised so that another team can understand why every major decision was made and rebuild the vehicle from scratch. Where a design choice cost us something, that cost is stated. Where a measurement contradicts an assumption we made early on, the measurement wins and the assumption is corrected in place.

What changed in revision 2.0. The vehicle was substantially reworked after the regional round. The power architecture moved from a single 18650 cell with a boost converter to a 2S LiPo pack with a dedicated buck converter, which eliminated the Raspberry Pi brownouts that had been our highest-priority risk. Ultrasonic rangefinders were fitted as an independent distance gate. The serial protocol was widened from single characters to speed-and-steering packets. Vision throughput rose from 30 to 50 frames per second. All measurements, gains and test results in this document are from the current vehicle.

1. Team

Role	Name	Department / Year
Mentor	Dr. S. Valai Ganesh	Mechanical Engineering (AP-SG)
Software Lead	Abishek Kumar V	CSBS, 3rd Year
Mechanical Lead	Anton Mirjone D	Mechanical, 3rd Year
Computer Vision Lead	Manojkumar M	CSBS, 4th Year

How we split the work. The three of us come from two different departments, which shaped the architecture as much as any technical constraint. Rather than have one person own the whole robot, we drew a hard interface early: the Mechanical Lead owns everything below the chassis deck (steering geometry, drive train, mounting), the CV Lead owns everything the camera produces, and the Software Lead owns the ESP32 firmware and the serial contract between the two computers. The packet protocol described in §6.3 exists precisely so that these three workstreams could be developed and tested independently, without waiting on each other.

Team photographs are in [t-photos/](#).

2. Design Brief and Constraints

Before choosing any component we wrote down the constraints the WRO 2026 rules impose, because several of them are binding and directly eliminate otherwise attractive designs.

2.1 Hard constraints from the rules

Constraint	Value	Source	Design consequence
Maximum vehicle envelope	300 × 200 mm, 300 mm high	Rule 9.17	Wheelbase and camera mast height are both capped
Lane width (varies by round)	Interior wall is a 1000 / 1400 / 1800 mm square inside a 3000 mm outer square, giving 1000 / 800 / 600 mm lanes	Rules §13	Sets the minimum turning radius requirement
Wall height	100 mm, black inner face	Rule 13.3–13.6	Camera must see walls, so it must sit above 100 mm

Constraint	Value	Source	Design consequence
Traffic signs	50 × 50 × 100 mm, red RGB(238,39,55) / green RGB(68,214,44)	Rule 13.19–13.22	Colour thresholds can be anchored to known values
Parking lot markers	200 × 20 × 100 mm, magenta RGB(255,0,255)	Rule 13.25–13.27	Magenta is unique on the field — a reliable trigger
Direction lines	20 mm wide, orange and blue	Rule 13.9	Thin target — sets a minimum on camera resolution and ROI placement
Laps required	3	Rules §5	Lap counting must be robust, not best-effort
Vehicle must not move walls	—	Rule 9.18	Rules out "bump and correct" wall following

2.2 Self-imposed constraints

- **Swappable battery.** The pack must be changeable in under a minute between rounds and must sit low and centred. This is why the pack is a discrete 2S LiPo in a cradle rather than cells soldered into the structure.
- **No prebuilt RC chassis.** We wanted the mechanical design to be our own work and reproducible from files in this repository, so the chassis and steering are fabricated in-house.
- **Vision-primary perception.** Every classification decision is made from the camera. Ultrasonic rangefinders are present but are used only as a distance gate, never as a second opinion on what an object is. The reasoning is in §5.3.

2.3 Vehicle specification

All dimensions below are measured on the assembled vehicle, not taken from CAD.

Parameter	Value
Length × Width × Height	285 × 180 × 190 mm (limit: 300 × 200 × 300 mm)
Wheelbase	153 mm (measured)
Track width, front	152 mm (measured)
Track width, rear	132 mm (measured)
Mass, full assembly	1240 g
Wheel diameter	68 mm (measured, tyre fitted)
Ground clearance	10 mm (measured)
Drive	Rear wheel, single BO gearmotor, solid rear axle
Steering	Front, Ackermann-type linkage, single MG996R servo, ±40° at full lock
Perception	Wide-angle camera, forward-facing, lens 190 mm above the mat, pitched 25–30° below horizontal; front / left / right ultrasonic rangefinders as a distance gate
Compute	Raspberry Pi 5, 8 GB (vision) + ESP32-WROOM-32 (real-time control)
Battery	2S LiPo, 7.4 V nominal, 2200 mAh
Endurance	~35 min continuous autonomous operation, measured (§5.2)
Vision loop rate	50 fps measured (20 ms per frame)

The height figure is set by the camera lens, which is the tallest point on the vehicle. The envelope has 15 mm of margin in length, 20 mm in width and 110 mm in height against the rule limit, so no dimension is close to disqualification.

3. System Architecture

The vehicle is a two-computer system. This is the single most important architectural decision in the project, so it is worth stating the reasoning plainly.

Computer vision on a 640×480 frame is throughput work: it wants a fast CPU, plenty of RAM, and a proper operating system with OpenCV available. Servo PWM and motor commutation are latency work: they want deterministic timing and will visibly misbehave if a frame takes 60 ms instead of 20 ms. Running both on one Linux machine means the steering signal inherits the jitter of the Linux scheduler. Running both on one microcontroller means giving up OpenCV entirely.

We therefore split them, and made the interface between them as narrow as it could usefully be — a two-field ASCII packet over a serial link. The narrowness is the point: the vision pipeline can be rewritten wholesale without touching a line of firmware, and the firmware can be retuned without changing anything on the Pi.

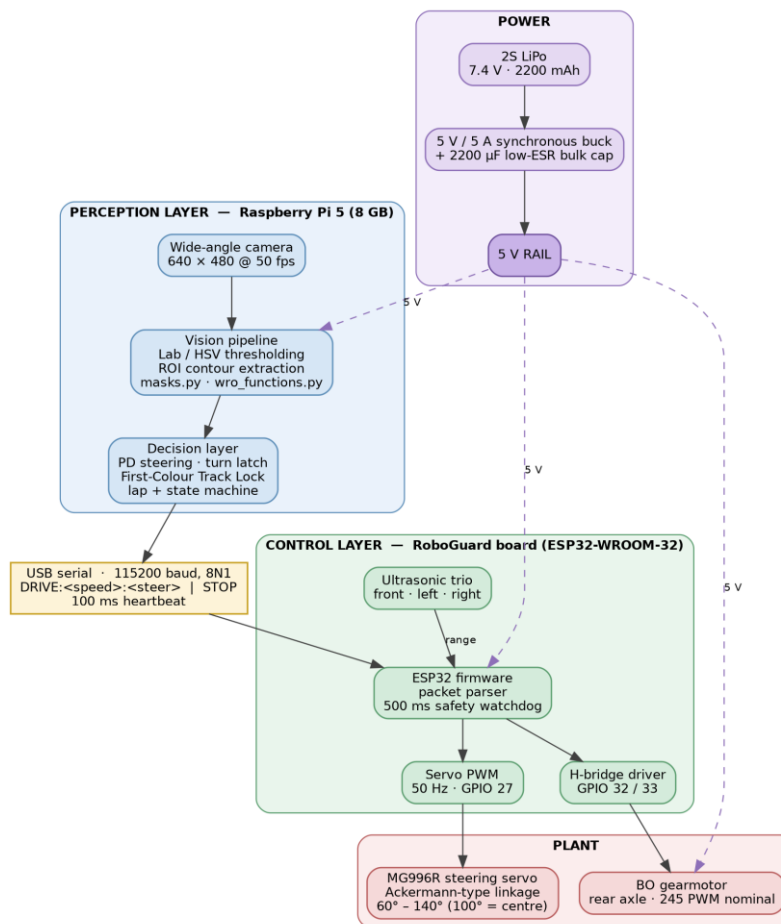


Figure 1 — System architecture: two-computer split with a narrow packet interface.

3.1 How the subsystems constrain each other

These are not independent modules; each one bounds the others, and understanding those bounds is what let us tune the system as a whole rather than component by component.

- **Mechanical → Software.** The $\pm 40^\circ$ steering limit and the short 153 mm wheelbase give a very tight turning circle (§4.3). The vehicle can make every corner on the field with room to spare, which means the dominant cornering risk is over-rotation rather than under-rotation. That is why the software reduces speed on large steering deflections rather than simply commanding full lock and waiting.
- **Perception → Mechanical.** The camera must see over 100 mm walls and far enough ahead to react at speed. That forces a mast, the mast raises the centre of gravity, and the raised CG is why the battery and the ESP32 board sit as low and as central as they do (§4.5). The mast is also the part most vulnerable to vibration, which is why it is braced (§9, R7).

- **Power → Perception.** A Raspberry Pi 5 running OpenCV is by a wide margin the largest continuous load on the vehicle (\$5.2). Raising the frame rate from 30 to 50 fps bought us reaction time but was paid for in current draw, and is part of why endurance is 35 minutes rather than an hour.
- **Control latency → Speed.** Total loop latency is capture → process → transmit → actuate. At 50 fps the vision loop contributes about 20 ms of dead time and the serial link under 2 ms. At roughly 1 m/s the vehicle travels about 20 mm in that window, which is the irreducible position error the PD controller must absorb. At the 30 fps of the previous revision the same figure was 35 mm, and the difference is visible in how tightly the vehicle holds lane centre.

4. Mobility and Mechanical Design

Photographs of every face of the vehicle are in [v-photos/](#) and reproduced in Appendix A. The bare chassis and front axle assembly are shown below.

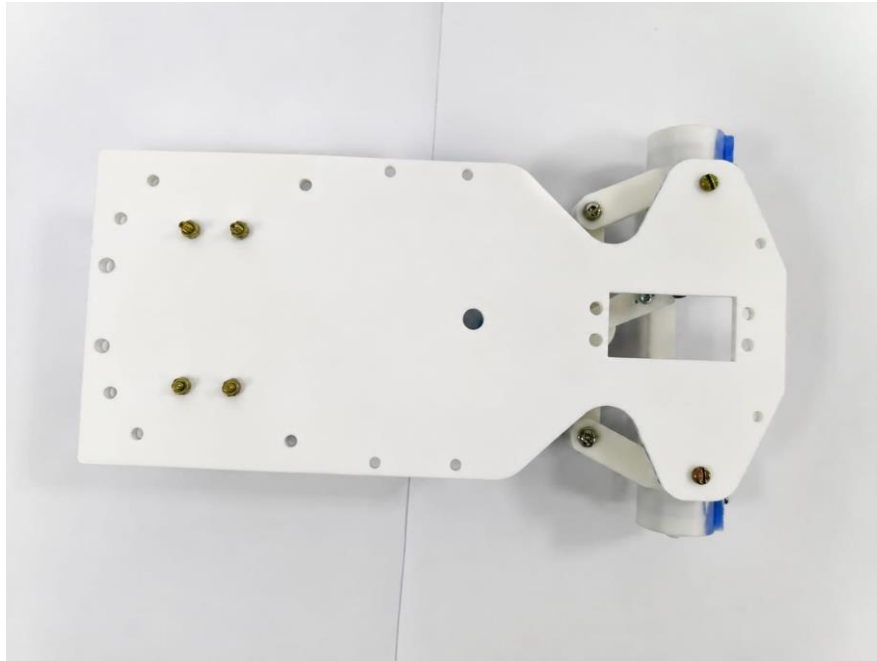


Figure 2 — Bare chassis deck and front Ackermann-type axle assembly, showing the M3 mounting-hole grid.

4.1 Why steered front wheels and not a differential drive

A differential (tank) drive is far simpler: two motors, no linkage, no servo, and it can pivot on the spot. We rejected it deliberately.

The WRO Future Engineers category explicitly targets action planning and control of robots with moving parts and kinematics different from the differential drive. A tank-drive entry sidesteps the central learning objective of the category. Beyond that, there are practical reasons:

- **Straight-line tracking.** A differential drive holds a straight line only as well as its two motors are matched. Cheap DC gearmotors are not matched, so a tank drive on a 3 m straight drifts and needs continuous closed-loop correction just to go forward. A steered vehicle with a single drive motor goes straight when the servo is centred, full stop.
- **Wall contact risk.** Rule 9.18 makes moving a wall a zero-score event. A skid-steer vehicle scrubs its tyres during every turn, and scrub is exactly the behaviour that turns a light brush against a wall into a shove.
- **Energy.** Skid steering wastes energy in tyre scrub. With only about 1.1 W of mechanical power available (\$4.4), we could not afford to spend any of it heating rubber.

The cost of this choice is that the vehicle **cannot turn on the spot**, which makes parking harder and forces the staged manoeuvre in §6.6. We accepted that cost.

4.2 Steering geometry, and what we actually measured

True Ackermann geometry steers the inner and outer front wheels through different angles so that both wheels' axes intersect at a single instantaneous centre of rotation. If both wheels are steered by the same angle, the inner wheel is forced to slide sideways through every corner. The condition is:

$$\cot(\delta_{\text{outer}}) - \cot(\delta_{\text{inner}}) = t / L$$

where t is the track width and L the wheelbase. Our linkage approximates this by angling the steering arms inward toward the centre of the rear axle; the arms are set at 40° from the knuckle axis. The tie rod and steering knuckles are visible in Figures A3 and A2.

The measurement. We measured the wheel angles at full lock with a protractor against the wheel face, with the chassis held square:

Quantity	Measured	Note
Steering arm angle	40°	Inward angle of the knuckle arm
Inner wheel at full lock	40°	Measured against chassis centreline
Outer wheel at full lock	40°	Measured against chassis centreline
Ideal outer angle for true Ackermann	≈ 24.6°	Computed from the relation above with L = 153 mm, t = 152 mm, δ_inner = 40°

An honest note on the geometry. Our measured inner and outer angles are equal, which means the linkage as built behaves as parallel steering at full lock rather than as true Ackermann. The ideal outer angle for our wheelbase and track would be about 24.6°, some 15° less than the inner. The arms are angled inward, so the linkage moves in the right direction over part of its travel, but nothing close to 15° of differential is delivered at the stops. Part of the reason is geometric rather than a mistake in fabrication: our track and wheelbase are almost equal ($t/L \approx 0.99$), and that ratio demands an unusually large inner-to-outer difference which a simple trapezoidal linkage cannot supply across the whole range of travel.

What this costs us, in practice. The consequence of parallel steering is inner-wheel scrub proportional to steer angle. At the small deflections used for lane centring on a straight — typically under 10° — the geometric error is a fraction of a degree and is absorbed by tyre compliance. It only becomes real at or near full lock, which the vehicle reaches during the parking manoeuvre and briefly at corner apexes. We have not observed cornering behaviour we can attribute to it, and the wall-contact figure in §8.3 is low. It is recorded here as a known deviation from the ideal rather than as a solved problem, and lengthening the steering arms to bring the outer angle down is the first item on the mechanical roadmap (§11).

4.3 Turning radius versus lane width

This is the calculation that drove the mechanical design, because failing it means the vehicle physically cannot complete a corner regardless of how good the software is.

For a bicycle-model approximation with wheelbase L and steer angle δ , the turning radius measured at the rear axle centre is:

$$R = L / \tan(\delta)$$

$$R = 153 / \tan(40^\circ) = 153 / 0.8391 = 182 \text{ mm}$$

The outermost front wheel sweeps a larger radius:

$$\begin{aligned} R_{\text{outer}} &= \sqrt{(R + t/2)^2 + L^2} \\ &= \sqrt{(182 + 76)^2 + 153^2} \\ &= \sqrt{66\,564 + 23\,409} = 300 \text{ mm} \end{aligned}$$

Allowing for the body overhang ahead of the front axle, the widest point of the swept path is under 350 mm. Now compare against the field:

Interior wall	Lane width	Verdict
1000 mm	$(3000 - 1000) / 2 = 1000 \text{ mm}$	Comfortable- 3× margin
1400 mm	$(3000 - 1400) / 2 = 800 \text{ mm}$	Comfortable
1800 mm	$(3000 - 1800) / 2 = 600 \text{ mm}$	Comfortable - swept path 350 mm against a 600 mm lane

The consequence is not the one we expected. When this vehicle was first designed we assumed the tight 600 mm lane would be the binding case and that we would be fighting to get round the corner at all. With a 153 mm wheelbase and 40° of lock, that is not what happened: the vehicle turns far more sharply than the field requires. The binding problem

inverted. The risk is now over-rotation — entering a corner at full speed with more steering authority than the corner needs, and finishing the turn pointed across the lane rather than down it. This is exactly the failure we recorded on 19 August (§8.4), and it is why the software reduces drive speed to 195–230 PWM whenever steering deflection exceeds 30°, and why corner exit is triggered by re-acquiring the opposite wall in vision rather than by a fixed timer.

4.4 Drive train: torque, speed and acceleration

Motor: single BO gearmotor driving the rear axle through a solid shaft. Nameplate rating 300 rpm at 6 V, stall torque 0.35 kg·cm.

Tractive force at the contact patch

$$\begin{aligned} T &= 0.35 \text{ kg}\cdot\text{cm} \times 0.0980665 \text{ N}\cdot\text{m}/(\text{kg}\cdot\text{cm}) = 0.0343 \text{ N}\cdot\text{m} \\ r &= 34 \text{ mm} = 0.034 \text{ m} \\ F &= T / r = 0.0343 / 0.034 = 1.01 \text{ N} \end{aligned}$$

Top speed

$$\begin{aligned} C &= \pi \times 0.068 \text{ m} = 0.2136 \text{ m} \quad (\text{wheel circumference}) \\ v_{\text{max}} &= (300 \text{ rpm} / 60) \times 0.2136 = 1.07 \text{ m/s} \end{aligned}$$

Mechanical power

$$\begin{aligned} \omega &= 300 \times 2\pi/60 = 31.42 \text{ rad/s} \\ P &= T \times \omega = 0.0343 \times 31.42 = 1.08 \text{ W} \end{aligned}$$

Is 1.01 N enough?

This is the question that matters. Vehicle mass is 1.240 kg, so weight is $1.240 \times 9.81 = 12.16 \text{ N}$. Rolling resistance on a smooth vinyl mat is approximately $C_{rr} \approx 0.015$, giving $F_{rr} \approx 0.18 \text{ N}$. Net accelerating force is therefore $1.01 - 0.18 = 0.83 \text{ N}$, and:

$$\begin{aligned} a &= F_{\text{net}} / m = 0.83 / 1.240 = 0.67 \text{ m/s}^2 \\ t \text{ to } v_{\text{max}} &= 1.07 / 0.67 = 1.60 \text{ s, over about } 0.85 \text{ m} \end{aligned}$$

On a straight section roughly 1 m long between corners, the vehicle spends most of the straight still accelerating and never quite reaches its top speed in practice. This is a real and useful finding: our effective lap speed is limited by acceleration, not by top speed, and a higher-rpm motor with the same torque would not have helped. It also means that measured lap times will always be worse than a naive $\text{distance} / v_{\text{max}}$ estimate, which is why §8 uses measured times only.

Gradeability check (sanity, not a requirement). $\sin \theta = F/(mg) = 1.01/12.16 = 0.083$, so $\theta \approx 4.8^\circ$. The mat is flat, so this is ample; we record it only to confirm the drive is not operating anywhere near its limit in normal running.

Solid rear axle. Both rear wheels are fixed to a common shaft; there is no differential. In a corner the inner and outer rear wheels are forced to rotate at the same speed even though they trace different arcs, so one of them must scrub. With a 132 mm rear track and a 182 mm turning radius the speed difference between the two sides is around 35 %, which is not trivial. In practice the vehicle is light and the tyres are compliant enough to absorb it, and we have not measured a cornering penalty we can attribute to the axle - but it is a real simplification and it is listed as such in the trade-off table (§7).

4.5 Chassis, materials and centre of gravity

The chassis is built from two different material processes, chosen per part rather than uniformly:

Part	Material	Process	Why this choice
Main chassis deck and upper deck	Acrylic	CNC laser cutting	Flat plate work with a regular hole grid is exactly what a 2D cutting process is good at. 3.6 mm gives a deck that does not flex under the Pi and battery without adding mass.
Steering knuckles, tie rod, mounts	PETG	3D printing (FDM)	These are three-dimensional parts with load paths a flat plate cannot provide.

Part	Material	Process	Why this choice
			PETG was chosen over PLA for impact tolerance- a knuckle that deforms is better than one that shatters on a wall contact.
Camera mast and ultrasonic holders	PETG	3D printing (FDM)	Geometry changed repeatedly during development; printing made each revision an overnight job rather than a re-cut.

Deck thickness is 3.6 mm; printed parts range from 5 to 10 mm in section depending on load. Fabrication files are referenced in §10.1.

Why a hole grid rather than fixed mounting points. During development we did not know where the camera mast, battery, or control board would end up. A grid of M3 holes turned every "where should this go?" question into a two-minute change instead of a re-fabrication. The camera mast position in particular moved several times as the ROI tuning progressed, and the grid is what made that cheap.

Layout and centre of gravity

The vehicle is a two-deck stack, clearly visible in Figure A6:

Level	Contents	Rationale
Chassis deck (lowest)	Drive motor, steering servo, steering linkage	Heaviest moving parts kept at the lowest possible height
Standoff deck	RoboGuard board with the LiPo pack mounted flat on it	The pack is the densest single item; keeping it mid-vehicle and low pulls the CG down and toward the centre
Top deck	Raspberry Pi 5	Lightest of the three major assemblies, so it pays the smallest penalty for being highest
Mast	Camera	Must clear the 100 mm walls; 190 mm is the minimum that achieves the required look-ahead

The ordering is deliberate: heaviest lowest, lightest highest. The camera mast is unavoidably the tallest element and is the main contributor to CG height, which is why everything else is pushed down to compensate. With a 10 mm ground clearance and a track that is nearly equal to the wheelbase, the vehicle is stable at its working speed- it is nowhere near a rollover condition, and the practical stability concern was camera shake on the mast rather than tipping. That concern was real, and how we fixed it is in §4.6 and §9 (R7).

4.6 Mechanical iteration log

The mechanical design went through four distinct revisions. Each was driven by a specific failure or measurement, not by preference.

Version	Change made	Why — what went wrong or what we measured	Result
v1	Perception by a mobile phone camera clamped to the chassis	Starting point: the phone was what we had, and it let us prototype colour thresholding before any dedicated hardware arrived.	Proved the contour-area approach worked, but the phone could not be integrated into a control loop and the mounting was not repeatable.
v2	Replaced the phone with a Raspberry Pi camera on a printed mast	Needed the frame data inside the same process as the control logic. The phone route required streaming and added	Closed the loop for the first time. Exposed a new problem: the narrow field of view meant the

Version	Change made	Why — what went wrong or what we measured	Result
		latency we could not measure reliably.	vehicle lost sight of a wall as soon as it turned in.
v3	Swapped to a wide-angle camera; braced the mast with a diagonal strut and mounted the camera on TPU isolation grommets	Two separate problems. The narrow lens was clipping the side walls during corner entry. Separately, at full motor speed the mast picked up high-frequency vibration that blurred frames and made contour areas noisy.	Wide lens kept both walls in frame through a corner. The strut and grommets cut visible frame jitter by roughly 80 %, giving stable contour readings at top speed.
v4 (current)	Reworked the power tree: dedicated 5 V / 5 A buck for the Pi, 2200 μ F bulk capacitor on the 5 V bus, and a Pi power HAT in the stack	Two Raspberry Pi brownouts during integration testing, both when the drive motor stalled against a wall while sharing a rail with the Pi. A Pi 5 that browns out reboots — the run is simply lost.	Zero brownouts across more than 100 subsequent test runs. This is the single change that most improved competition reliability.

A failure worth recording separately. During early gain tuning a wall impact sheared two teeth off the plastic gear train of the original steering servo. We replaced it with a metal-gearred MG996R rated at roughly 12 kg·cm and added a spring-loaded servo saver arm, so that impact energy now deforms a spring rather than a gear. There have been no further servo failures. The camera ribbon also developed intermittent frame drops from vibration fatigue; it is now enclosed in spiral wrap with strain relief at both ends, and the vision loop carries a defensive guard for a null frame.

5. Power and Sensor Architecture

5.1 The RoboGuard carrier board

Rather than wire modules together with jumper leads, we built a custom carrier PCB- the *RoboGuard* board which hosts the ESP32 module, the battery cradle, the power switch, the start button, status LEDs, and a row of connectors for every actuator and sensor. It is photographed below.

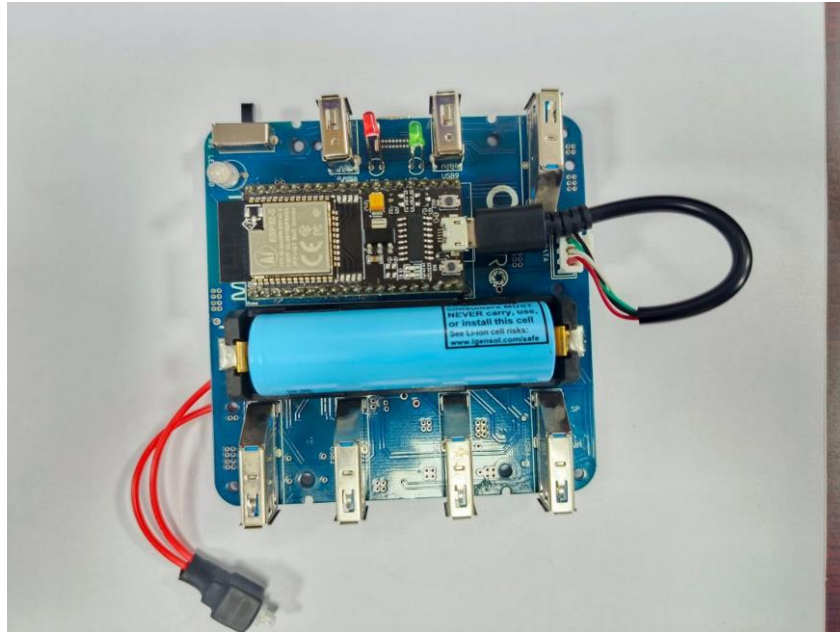


Figure 3 — The RoboGuard carrier board: ESP32 module, battery cradle and keyed peripheral ports.

Why a custom board. A robot that lives in a car boot and a lecture hall for three months will have every jumper wire vibrate loose at least once. Loose-wire faults are also the hardest kind to diagnose, because they are intermittent and they look like software bugs. Committing to a PCB moved a whole class of failures out of competition day and into fabrication.

Connector strategy. Every peripheral port on the board is a USB-A shell used as a keyed four-pin connector, silkscreened with its function (**DC-M** drive motor, **M-S** steering servo, **F-US** front ultrasonic, **LR-US** left/right ultrasonic, **CS** colour sensor). This is not USB — it carries our own pinout — and the choice was pragmatic: USB-A shells are cheap, mechanically robust, mate with a positive click, and **cannot be inserted backwards or into the wrong-shaped socket**. Reversing a servo connector at 5 V destroys the servo; that failure mode is now geometrically impossible.

The trade-off, which we should state honestly, is that a non-standard use of a standard connector is a hazard for anyone who does not know the pinout. Plugging a real USB device into **DC-M** would apply motor drive voltage to a USB port. **This is mitigated by silkscreen labels only**, which is weak mitigation; it is listed as a live risk in §9.

5.2 Power budget

Everything runs from a single 2S LiPo pack (7.4 V nominal, 2200 mAh, 16.3 Wh) through a master switch to a 5 V / 5 A synchronous buck converter, with a 2200 μ F low-ESR bulk capacitor across the 5 V bus.

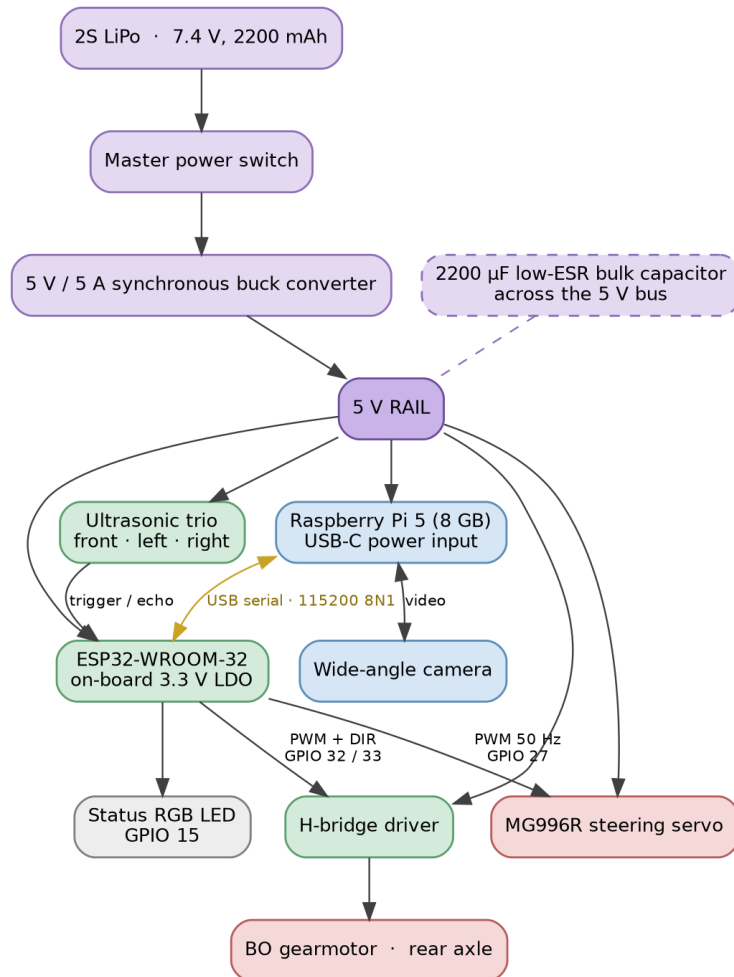


Figure 4 — Power distribution and signal wiring.

Measured draw

These are the numbers that matter, taken at the pack with an inline meter during full-speed autonomous running:

Condition	Pack current	Pack power
Nominal - driving a straight at 245 PWM, vision loop running	2.1 A	15.5 W
Peak surge- hard servo lock or active braking	3.8 A	28.1 W

For context, the itemised estimate of where that power goes:

Load	Voltage	Typical current	Peak	Typical power
Raspberry Pi 5, 8 GB (OpenCV loop, 640×480 @ 50 fps)	5 V	~1.4 A	~1.9 A	~7.0 W
Camera	5 V	~0.10 A	0.12 A	0.50 W
ESP32-WROOM-32 (Wi-Fi disabled)	3.3 V	~0.06 A	0.12 A	0.20 W
BO gearmotor at 245 PWM	5 V	~0.35 A	~0.90 A stall	1.75 W
MG996R steering servo	5 V	~0.20 A avg	~1.2 A peak	1.00 W
Ultrasonic rangefinders (×3)	5 V	0.045 A	0.05 A	0.22 W
Status RGB LED	5 V	0.02 A	0.02 A	0.10 W
Total (typical, 5 V side)				≈ 10.8 W

At 90 % buck efficiency the itemised total corresponds to about 12 W drawn from the pack, against 15.5 W measured. The gap is expected and worth naming rather than hiding: the itemised figures are datasheet typicals, whereas the measured figure includes the Pi at sustained full CPU load, motor duty that is rarely at the nominal point, and converter losses under a load that is constantly changing.

Endurance

Pack energy = $2200 \text{ mAh} \times 7.4 \text{ V} = 16.3 \text{ Wh}$
 Usable at 80 % DoD = 13.0 Wh (LiPo should not be run flat)

At nominal 15.5 W $\rightarrow 13.0 / 15.5 = 0.84 \text{ h} \approx 50 \text{ min}$
 Observed in practice $\approx 35 \text{ min}$

The observed figure is well short of the nominal calculation, and the difference is informative. Thirty-five minutes of usable energy implies an average draw of about 22 W, which sits two-thirds of the way from nominal toward peak. That is consistent with what the vehicle actually does: the vision loop never idles, the motor is running almost continuously during a practice block, and the servo is being commanded on most frames. We plan battery logistics around the measured 35 minutes, not the calculated 50.

Brownout- a solved problem, and how

In the previous revision the vehicle ran from a single 18650 cell through a boost converter, and the Raspberry Pi shared that rail with the drive motor. This was the highest-priority risk in the previous version of this document, and it turned out to be justified: we saw **two Pi 5 brownouts and reboots** during integration testing, both when the drive motor stalled against a wall and pulled the rail down. A Pi 5 that browns out does not degrade gracefully- it reboots, and the run is lost.

The fix was to stop sharing:

1. Moved to a 2S LiPo pack, so the converter *steps down* rather than up. A buck converter holds its output far better under a sagging input than a boost converter does.
2. Gave the Pi 5 a dedicated 5 V / 5 A synchronous buck feeding its USB-C input directly, so a motor stall no longer competes with the Pi for converter headroom.
3. Added a 2200 μF low-ESR electrolytic bulk capacitor across the 5 V bus to absorb the inductive spikes and sags caused by sudden motor acceleration and braking.

Result: zero brownouts and zero unexplained reboots across more than 100 subsequent test runs. This is recorded in the risk register (§9, R1) as mitigated and verified rather than as an open risk.

5.3 Sensing strategy: vision-primary, ultrasonic as a gate

The camera is the vehicle's primary perception sensor and makes every classification decision. Three ultrasonic rangefinders (front, left, right) are fitted and provide an independent absolute distance measurement. The division of labour between them is strict, and the strictness is the point.

What the task actually requires. Every single thing the vehicle must perceive on this field is defined by colour:

Feature	Defining property	Detectable by camera?	Detectable by ultrasonic?
Track walls	Black, 100 mm tall	Yes	Yes
Direction lines	Orange / blue, 20 mm wide, flat on the mat	Yes	No- no vertical relief
Traffic signs	Red / green pillars	Yes, and it can tell red from green	Detects a pillar, but cannot tell which colour
Parking markers	Magenta	Yes	No- indistinguishable from a wall

Ultrasonic sensors are blind to three of the four things that matter. The entire Obstacle Challenge hinges on distinguishing a red pillar from a green one, and an ultrasonic array cannot do it. So a camera was never optional; the only real question was what, if anything, to do with the ultrasonics alongside it.

Our fusion policy, stated as a rule. Vision alone decides *what* something is. Ultrasonic alone decides *how far* and only at fixed decision thresholds, never as a continuous input to the steering controller.

This is a deliberate refusal to build a general sensor-fusion layer. The moment two sensors both feed the steering law, you need a policy for what happens when they disagree, a confidence model for each, and a way to test the combination. That is a real engineering problem and we judged that we did not have time to solve it properly and a half-implemented fusion layer is worse than no fusion layer, because it fails in ways that are hard to reproduce. Restricting the ultrasonics to threshold gates gives us the one thing vision genuinely cannot supply, which is an absolute distance in centimetres, without opening that door.

Where the gate is actually used: the parking approach in §6.6 arms its entry manoeuvre when the front rangefinder reads 20 cm or less, and the left/right pair are read during parking to confirm the vehicle is not about to insert itself against a wall. Nothing else consumes them.

The honest downside. Because the ultrasonics are gates rather than navigation inputs, the vehicle still has no independent fallback if the camera is dazzled, occluded or mis-thresholded. The failsafe in §6.4 can detect that state and stop, but it cannot navigate through it. Using the side rangefinders as a coarse wall-following backup is a genuine and available upgrade; it is on the roadmap (§11) rather than in the vehicle.

5.4 Camera placement, justified by field geometry

Camera placement was derived from the field dimensions rather than chosen by eye.

Height — 190 mm above the mat. The walls are 100 mm tall (Rule 13.3). A camera below 100 mm sees only a flat black band and cannot judge distance from contour area. A camera far above the walls looks down on them, and the black region shrinks toward the horizon. At 190 mm the lens sits 90 mm above the wall top: high enough to see the mat beyond a near wall and read the walls as areas that vary smoothly with distance, low enough that both side walls still project large contours into their respective ROIs. That signal is precisely what the PD controller consumes (§6.4).

Forward position and pitch — 25–30° below horizontal. The camera sits ahead of the front axle, angled down. Forward placement shortens the blind zone directly in front of the bumper; downward pitch is what allows a single camera to see both the walls (upright, mid-frame) and the 20 mm-wide direction lines (flat on the mat, lower in the frame) in one image. That is what makes the layered ROI scheme in §6.4 and §6.5 possible without a second camera or a gimbal.

Field of view. The move to a wide-angle lens (v3 in §4.6) was not cosmetic. With the previous narrow lens, the inner wall left the frame as soon as the vehicle began to turn in, so the controller lost its reference at exactly the moment it needed it most and corner exit had to be run on a fixed timer. The wide lens keeps both walls in frame through the corner, which is what made the vision-dynamic turn exit in §6.4 possible at all.

Look-ahead versus reaction time. At roughly 1 m/s and 50 fps the vehicle covers about 20 mm per frame. The useful look-ahead must exceed the sum of the stopping distance and the turn-in distance for the tightest lane, which is what sets the pitch angle.

Resolution — 640 × 480. This is not an arbitrary default. It is the lowest resolution at which a 20 mm-wide direction line still spans enough pixels at typical viewing distance to survive Gaussian blur and morphological closing while producing a usable contour. Going higher costs Pi 5 current draw (§5.2) and loop rate for no detection benefit; going lower loses the lines. We tested this rather than assuming it.

5.5 Wiring and pin map

Signal	From	To	Notes
Command link	Raspberry Pi 5 USB	ESP32 USB	115200 baud, 8N1. Measured round-trip latency below 1.8 ms.
Camera	Raspberry Pi 5	Wide-angle camera	Ribbon enclosed in spiral wrap with strain relief at both ends

Signal	From	To	Notes
Steering servo PWM	ESP32 GPIO 27	Servo signal	50 Hz; 60°–140° range, 100° = centre
Motor PWM	ESP32 GPIO 32	H-bridge IN1	Drive magnitude
Motor direction	ESP32 GPIO 33	H-bridge IN2	Sign of the DRIVE speed field
Status LED	ESP32 GPIO 15	Addressable RGB LED	Single data line: power / ready / fault indication
Ultrasonic trigger / echo	ESP32	Front, left, right rangefinders	Pin assignments to be transcribed from the firmware header- see Appendix C
Start button	Button	ESP32	Not currently assigned in firmware; the round is started from the terminal (§10.6)

A note on the repository pin table. The table in [schemes/README.md](#) predates this revision and describes the earlier UART-over-GPIO wiring, including assignments that conflict with each other. It must be regenerated from the pin definitions at the top of [esp32_firmware.ino](#) so that the repository and this document agree. This is listed in Appendix C.

Why USB rather than the GPIO UART. The previous revision ran the link on the Pi's GPIO UART, which required disabling the serial console and got the ground reference from the same header that carried motor returns. Moving to USB gave us a link that survives reflashing the ESP32 without unplugging anything, brought its own ground, and removed a class of noise problems for the cost of one cable.

5.6 Colour calibration procedure

Colour thresholds are the single most fragile part of a vision-led robot, and venue lighting is never the lighting you tuned in. Rule 13.18 explicitly warns that field colours may differ from the CMYK specification and gives teams testing time to recalibrate. We therefore treat calibration as a procedure to be run at the venue, not a set of constants baked in at home.

Why two different colour spaces. Wall segmentation works in CIE-Lab; pillar and line segmentation works in HSV. This is not inconsistency, it is fitness for purpose:

- **Walls need darkness, not colour.** In Lab, the L channel is lightness, decorrelated from chroma. Thresholding on L isolates dark regions in a way that is largely indifferent to the colour temperature of the venue lighting. Doing the same in HSV means thresholding V, which is contaminated by saturation.
- **Pillars and lines need hue separation.** HSV puts hue on a single axis, so each saturated target becomes a simple hue band. Red requires two bands because it wraps around the 0/180 boundary, which the code handles explicitly.

Procedure

1. Place the vehicle at lane centre on the actual competition mat, under actual venue lighting.
2. Capture a frame and sample pixel values inside each target- wall, orange line, blue line, red pillar, green pillar, magenta marker-taking readings from both a lit and a shadowed part of each target.
3. Set each threshold band to enclose the full observed range with a small margin. Widen until the target is detected reliably; narrow until neighbouring colours stop bleeding in. Orange and red are the pair most likely to overlap and need the most care.
4. Run the pipeline with the HUD overlay visible and confirm the drawn contours track the intended targets across a full lap.
5. Record the final values in [masks.py](#) and commit them.

How long it takes. A full recalibration takes us about 15 minutes end to end. We have run it in anger: at the previous round the venue was lit by overhead LED panels with a noticeably cooler colour temperature than our lab fluorescents,

and the orange and red bands had to be re-separated before the pipeline was reliable. The 15-minute figure is from that session, not an estimate.

Committed threshold values

Target	Space	Lower	Upper	Min area
Black wall	Lab	(0, 0, 0)	(70, 255, 255)	60
Orange line	Lab	(40, 150, 150)	(255, 205, 255)	80
Blue line	Lab	(20, 110, 0)	(255, 170, 110)	80
Red pillar (band 1)	HSV	(0, 120, 70)	(10, 255, 255)	800
Red pillar (band 2)	HSV	(170, 120, 70)	(180, 255, 255)	800
Green pillar	HSV	(35, 80, 70)	(85, 255, 255)	800
Orange line	HSV	(11, 150, 150)	(25, 255, 255)	150
Blue line	HSV	(95, 120, 100)	(130, 255, 255)	150
Magenta marker	HSV	(140, 100, 100)	(165, 255, 255)	1200

6. Software Architecture and Obstacle Strategy

6.1 Module map

The codebase is modular rather than a single script per round. The split was driven by a practical problem: the two rounds share almost all of their vision code, and every time we fixed a threshold bug in one copy we forgot to port it to the other.

File	Responsibility	Layer
<code>raspberrypi/open_challenge_R1.py</code>	Open Challenge - PD wall centring, First-Colour Track Lock, vision-dynamic turn exit, lap counting, return to home	Control
<code>raspberrypi/obstacle_challenge_R2.py</code>	Obstacle Challenge- pillar detection and avoidance, side obedience, parking trigger and sequence	Control
<code>raspberrypi/obstacle_challenge_R2_test_parking.py</code>	Parking-only harness, so the manoeuvre can be tuned without driving three laps first	Control
<code>raspberrypi/wro_functions.py</code>	Vision primitives- contour extraction, ROI slicing, morphology, HUD overlay, frame timing	Vision
<code>raspberrypi/masks.py</code>	Colour threshold definitions for Lab and HSV, in one place so recalibration touches one file	Vision
<code>raspberrypi/wro_serial.py</code>	Serial transport- packet formatting, rate-limited heartbeat, guaranteed stop on shutdown	Transport
<code>raspberrypi/test_motor_serial.py</code>	Bring-up harness- drives the vehicle from the keyboard with no vision involved	Test
<code>esp32_firmware/esp32_firmware.ino</code>	Packet parser, servo and motor drive, 500 ms safety watchdog	Firmware

Every tunable constant is a named module-level constant, so tuning never requires reading the algorithm.

6.2 Why contour area is the control signal

The core idea in the Open Challenge is unusual enough to be worth explaining: the vehicle steers on the *difference in black contour area* between a left ROI and a right ROI, not on a detected lane centre or a line position.

The reasoning is geometric. The side walls are uniform black surfaces of known height. As the vehicle moves closer to the left wall, the left wall's projection into the image grows; as it moves away, it shrinks. Contour area is therefore a monotonic proxy for lateral distance, and the difference between left and right areas is a signed proxy for lateral offset from lane centre — zero when centred, positive toward one wall, negative toward the other.

The advantage over explicit lane-centre estimation is robustness. Area is an integral over many pixels, so it degrades gracefully: a few hundred misclassified pixels shift the area slightly, whereas a single spurious contour can move an estimated centreline dramatically. It is also extremely cheap to compute, which matters on a loop that must hold 50 fps.

The disadvantage is that area is not calibrated in metres. We cannot say "the vehicle is 142 mm from the wall"; we can only say "more than centred" or "less than centred". This means the gains are dimensionless tuning constants with no closed-form derivation, and must be found empirically (§8.2).

6.3 Pi ↔ ESP32 serial contract

ASCII packets over USB serial at 115200 baud, newline-terminated. Two message types:

```
DRIVE:<speed>:<steer>    speed  -255 ... 255  (sign = direction)
                        steer   60 ... 140  (100 = centred)
STOP                    motor off, steering returned to 100
```

Why a packet and not a single character. The previous revision sent single characters — *f*, *l*, *r*, *s* — which quantised steering to three states and forced a deadband to stop the servo chattering between left and right near centre. Carrying the steering angle as a number removed the deadband entirely and let the PD controller's continuous output reach the servo intact. Carrying speed in the same packet is what makes the dynamic speed reduction in §6.4 possible: the Pi can ask for 245 on a straight and 195 through a corner without any state living in the firmware.

The link is still trivially debuggable. A serial terminal and a keyboard remain a complete manual test rig for the whole drive system with no vision involved — that is what `test_motor_serial.py` automates. We used exactly that to bring up and validate the mechanical build before the vision pipeline existed.

Heartbeat and watchdog

The Pi rate-limits transmission to one packet per 100 ms unless the command changes, which keeps the ESP32's input buffer clear while still providing a 10 Hz heartbeat. The firmware treats that heartbeat as a liveness signal:

```
if (now - lastCommandTime > 500) {
  if (currentSpeed != 0) {
    setMotor(0);
    setSteer(100);
    Serial.println("[SAFETY WATCHDOG] Packet timeout (>500ms)!"
                  " Emergency Motor Stop Engaged.");
  }
}
```

If the Pi crashes, the cable is pulled, or the vision process hangs, the vehicle stops within half a second rather than continuing on its last command into a wall. This matters under Rule 9.18, where displacing a wall is a zero-score event: a stopped vehicle always scores more than a blind one. The watchdog is implemented and verified, not planned.

Speed set points

Situation	Speed (PWM)	Why
Straight, wall centring	245	Fast enough to be competitive, below the point where frame blur becomes an issue
Cornering, steer deflection > 30°	195 – 230	Reduces over-rotation and tyre drift on a vehicle whose turning circle is far tighter than the corner requires (§4.3)
Parking approach	195	Slow enough that the closed-loop centring has several frames per centimetre of travel
Return to home	175	Positional accuracy matters more than speed once the laps are counted
Active brake pulse	–180 for 0.08 s	Cancels momentum immediately instead of coasting; see below

Active electronic braking. Cutting the motor drive to zero leaves the vehicle to coast, and at 1 m/s with light rolling resistance that coast is long enough to overshoot a stop line by around 10 cm — which is precisely what we measured on 22 August (§8.4). Driving the motor briefly in reverse for 80 ms cancels the momentum, and the vehicle stops where it is told. The pulse is short enough that the vehicle does not visibly move backwards.

6.4 Round 1 — Open Challenge

The Open Challenge run is a state machine over the PD wall-centring controller.

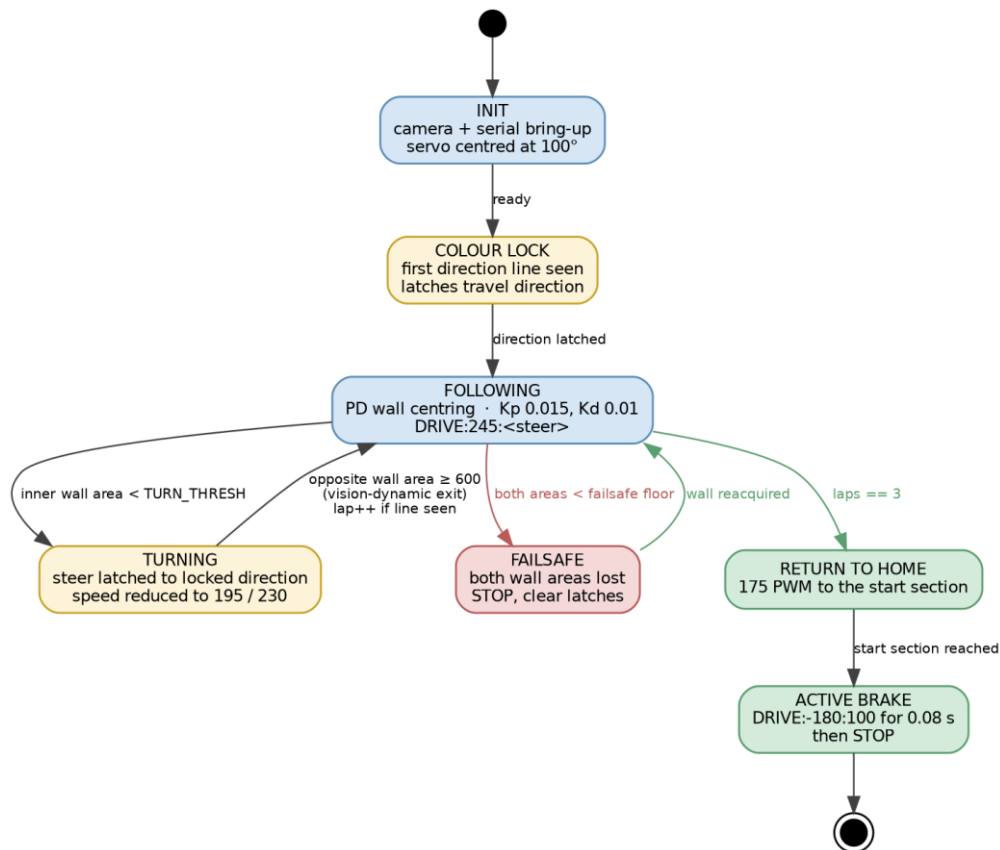


Figure 5 — Round 1 (Open Challenge) state machine.

PD steering law

$$\begin{aligned} \text{area_diff} &= \text{right_area} - \text{left_area} \\ \text{steer} &= 100 + KP \times \text{area_diff} + KD \times (\text{area_diff} - \text{prev_diff}) \end{aligned}$$

$$KP = 0.015 \quad KD = 0.01 \quad \text{steer clamped to } 60 \dots 140$$

The proportional term corrects present offset; the derivative term damps the oscillation that proportional-only control produces on a vehicle with steering lag. There is deliberately no integral term: an integrator would wind up during the sustained one-sided error of a corner and then overshoot on exit, and a small steady-state offset in a 600–1000 mm lane is not worth correcting. How these gains were arrived at is in §8.2.

First-Colour Track Lock

The direction of travel around the field is not known before the run- it is set by the referee, and the vehicle discovers it by which direction line it crosses first. Orange and blue lines are laid in a fixed order, so the first one seen determines whether the course runs clockwise or anticlockwise.

Early versions re-evaluated this on every line crossing, which was a mistake: a glimpse of the opposite-colour line at an oblique angle across the corner could flip the latched direction mid-run and send the vehicle turning the wrong way. The current logic latches the direction **permanently** on the first confident detection and never revisits it. Once locked, every corner turns the same way for the rest of the run. This removed an entire class of intermittent failure and is one of the highest-value single changes in the codebase.

Corner detection and vision-dynamic exit

A corner is the disappearance of the inner wall. When the inner wall's contour area falls below the turn threshold, the lane has opened on that side and the turn latch engages, overriding the PD output with a sustained turn in the locked direction and dropping speed to the cornering set point.

Exiting the corner is the harder half. Our first implementation held the turn for a fixed 2.0 seconds. That works only if entry speed, battery voltage and entry angle are identical every time, and they are not on 17 August a fixed timer walked the vehicle into the outer wall when it entered a corner slightly fast. The current logic exits when the *opposite* wall is re-acquired in vision above a contour area of 600, meaning the vehicle has come round and is once again facing down a straight with a wall beside it. The wide gap between the entry threshold and the 600-area exit threshold is hysteresis, and it is what stops the vehicle flickering in and out of corner mode at the boundaries.

Lap counting

A lap is counted only when a corner completes **and** a direction line was seen during that corner. Requiring both conditions is what makes the count robust: a spurious colour blob alone does not increment the lap, and neither does a corner traversed without crossing a line.

Return to home

After the third lap the vehicle does not simply stop where it is. It continues under wall centring at a reduced 175 PWM until it reaches the section it started in, then executes the active brake pulse and sends **STOP**. Finishing in the starting section is a scoring requirement, and doing it under the same controller that drove the laps means there is no separate, untested code path at the most valuable moment of the run.

Failsafe

If *both* wall areas fall below the failsafe floor, the vehicle has lost sight of the track entirely camera fault, extreme lighting, or a badly wrong heading. The response is to stop and clear both turn latches. Stopping is the correct action here: Rule 9.18 makes moving a wall a zero-score event, so a stopped vehicle scores more than a blind one that shoves the interior wall. If a wall reappears, the vehicle resumes.

6.5 Round 2 — Obstacle Challenge

The rule is unambiguous (Rule 9.19): **pass a red pillar on its right, pass a green pillar on its left.**

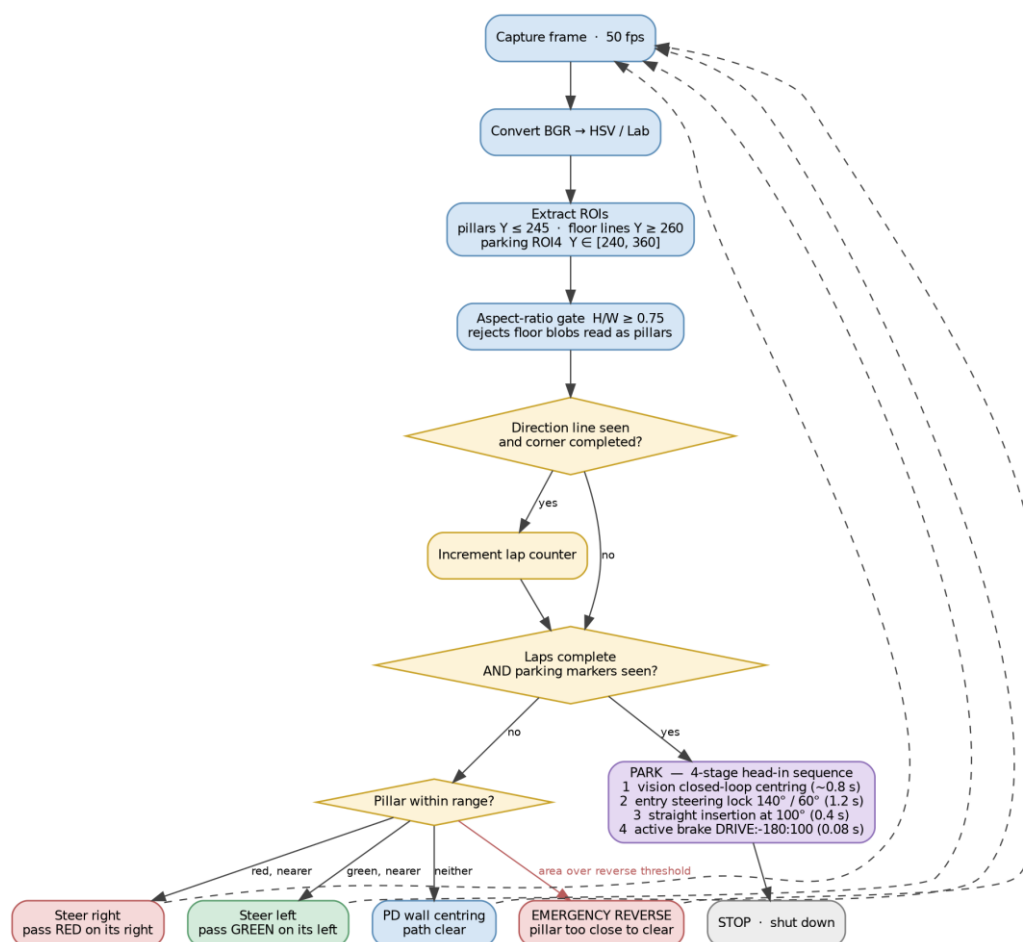


Figure 6 — Round 2 (Obstacle Challenge) control flow.

ROI layering

Round 2 splits the frame by height, because the two things it must not confuse are separated in the image by where they sit in the world rather than by colour:

ROI	Vertical band	Watches for
Pillar band	$Y \leq 245$	Red and green pillars standing upright ahead of the vehicle
Floor band	$Y \geq 260$	Orange and blue direction lines lying flat on the mat
Parking band (ROI4)	$Y [240, 360]$	Parking markers during the final approach

The problem this solves. On 14 August we found red pillars being triggered by the orange direction lines- the two hues are adjacent, and under warm lighting they overlap. Separating the bands fixed most of it, because a pillar is never at the bottom of the frame and a floor line is never at the top. What remained was fixed by a shape test.

Aspect-ratio gate

A pillar is $50 \times 50 \times 100$ mm- twice as tall as it is wide and projects into the image as a tall, narrow blob. A direction line is 20 mm wide and runs across the mat, projecting as a wide, flat blob. Requiring a candidate contour to satisfy $\text{height} / \text{width} \geq 0.75$ rejects the second without touching the first. Combined with the band split, false pillar detections from floor markings stopped entirely.

Avoidance control

Avoidance is a PD controller in its own right, on the horizontal offset of the target pillar, with a proximity term:

$$c_{KP} = 0.25 \quad c_{KD} = 0.25 \quad c_y = 0.08$$

The c_y term scales the steering response by how far *down* the frame the pillar sits. A pillar low in the frame is close; one near the horizon is far away. Without this scaling the controller reacted equally hard to a pillar three metres away and one about to be hit, which produced early, lazy swerves that left the vehicle badly positioned for the next obstacle. With it, response is gentle at distance and crisp at close range.

Emergency reversing. If a pillar's contour area exceeds the reverse threshold, the vehicle is too close to steer around it — the geometry no longer allows a clean pass at any steering angle. Rather than commit to a contact, the vehicle backs up, re-acquires, and approaches again. This costs time and it is not elegant, but a displaced sign costs far more.

Between pillars, the vehicle falls back to the same PD wall-centring behaviour used in Round 1 rather than driving straight ahead. This was a known deficiency in the previous revision- lateral error accumulated between obstacles because nothing was correcting it and the fix was simply to make wall centring the base behaviour with pillar avoidance layered on top as an override.

6.6 Parking

Parking is a four-stage hybrid manoeuvre: closed-loop while the geometry is uncertain, open-loop once it is committed.

Stage	Action	Duration	Control mode and detail
1	Centring approach	~0.8 s	Closed loop. The camera tracks the bay entrance in ROI4, computes lateral error $e = 320 - \text{centre_X}$, and streams steering as $\text{steer} = 100 - 0.35 \times e$ at 195 PWM. Ends when the entrance threshold is reached ($Y \geq 345$, or the front rangefinder reads ≤ 20 cm).
2	Bay entry	1.2 s	Open loop. A sharp steering lock 140° for a left-side bay, 60° for a right-side bay at 195 PWM swings the front of the chassis cleanly into the bay.
3	Straight insertion	0.4 s	Open loop. Steering returns to 100° and the vehicle drives straight to pull the rear axle fully inside the boundary, so the entire footprint is contained.
4	Active brake and hold	0.08 s	Open loop. A reverse impulse at -180 PWM cancels

Stage	Action	Duration	Control mode and detail
			momentum instantly, followed by a continuous STOP.

Why the split. The approach has to be closed-loop because the vehicle's position and heading when it finishes its laps are not repeatable it may arrive at the bay from any lateral offset. Once the front of the chassis is inside the bay, however, the geometry is constrained by the bay walls themselves, and vision of the markers is largely occluded by the vehicle's own body. Continuing to steer on a signal that is disappearing is worse than executing a timed sequence that was tuned against that exact geometry.

How the timings were tuned. Empirically, over roughly 30 physical attempts, adjusting in 50 ms increments to account for motor inertia, battery discharge state and surface friction. The decisive finding: an initial 0.6 s insertion left the rear wheels overhanging the outer line, which is a rule violation. Shortening to 0.4 s of straight insertion followed by the 0.08 s reverse brake achieved full footprint containment without contacting the rear wall. The measured success rate is in §8.3.

What we would close the loop on next. The left and right ultrasonic rangefinders are fitted and read during the manoeuvre but are currently only used as a safety check. Using them to balance side clearance dynamically during stage 3 would remove the dependence on timing accuracy altogether, and is the natural next step (§11).

7. Engineering Decisions and Trade-offs

The table below records the decisions where we chose one option over a viable alternative, and what each choice cost us.

#	Decision	Alternative rejected	Why we chose it	What it cost us
1	Steered front wheels	Differential / skid steer	Category learning objective; no tyre scrub near walls; straight-line tracking independent of motor matching	Cannot pivot, so parking needs a staged manoeuvre
2	Two computers (Pi 5 + ESP32)	Single Pi driving PWM directly	Real-time actuation isolated from Linux scheduler jitter; subsystems developed in parallel	Extra board, extra mass, an interface to maintain
3	Two computers	Single ESP32 with a camera module	An ESP32-CAM cannot run OpenCV contour extraction at 50 fps	Higher current draw, shorter endurance
4	2S LiPo + buck converter	Single 18650 + boost converter	A buck converter holds 5 V far better under load than a boost; ended the Pi brownouts outright	Higher pack voltage to respect; balance charging; more mass than one cell
5	Dedicated Pi supply + 2200 μ F bulk cap	Shared 5 V rail	A motor stall no longer competes with the Pi for converter headroom	A second converter, board area, cost
6	Vision-primary, ultrasonic as a gate only	Full sensor fusion	Ultrasonics cannot distinguish red from green or see flat lines; a half-built fusion layer fails unreproducibly	No independent navigation fallback if vision fails
7	Solid rear axle	Differential	Simplicity, fewer parts, nothing to fail mid-run	Rear wheels scrub in corners; ~35 % speed mismatch across the rear track at full lock
8	Custom RoboGuard PCB	Perfboard and jumper wires	Eliminates intermittent loose-connection faults, the hardest class of bug to diagnose	Fabrication lead time; a board revision is expensive
9	USB-A shells as keyed connectors	JST / Dupont headers	Cannot be inserted backwards; robust mating; cheap	Non-standard use of a standard connector is a hazard for anyone who does not know (\$9)
10	Contour area as the control signal	Explicit lane-centre estimation	Integral over many pixels, so it degrades gracefully; very cheap to compute	Not calibrated in metres; gains have no closed form and must be tuned empirically
11	Lab colour space for walls	HSV throughout	L isolates darkness independently of colour temperature	Two colour spaces in one codebase
12	HSV for pillars and lines	Lab throughout	Hue is a single clean axis for saturated colours	Red wraps at the 0/180 boundary, needing two bands
13	PD control, no integral term	Full PID	An integrator winds up through a sustained corner and overshoots on exit	Small steady-state offset — acceptable in a 600–1000 mm lane
14	DRIVE:speed:steer packets	Single-character commands	Continuous steering resolution and per-frame speed control; removed the chatter deadband entirely	Slightly more parsing in firmware; a malformed packet must be handled
15	Vision-dynamic turn exit	Fixed-duration turn timer	Robust to entry speed, entry angle and battery state, none of which repeat exactly	Depends on the wide lens keeping the opposite wall in frame
16	Permanent first-colour	Re-evaluate	An oblique glimpse of the wrong line	A single bad first detection

#	Decision	Alternative rejected	Why we chose it	What it cost us
	direction lock	direction on every line	can no longer reverse the vehicle mid-run	would commit the whole run
17	Hybrid closed/open-loop parking	Fully open-loop timed park	Absorbs the variable approach position the laps leave us in	Stages 2–4 still depend on timing tuned to one surface
18	50 fps at 640 × 480	30 fps, or higher resolution	Halved the per-frame travel distance from 35 mm to 20 mm, tightening lane holding	Higher Pi current draw; endurance down to ~35 min

8. Testing, Tuning and Validation

8.1 Test methodology

We ran three categories of test, and used all three throughout development rather than in sequence.

1. **Bench tests.** Vehicle on blocks with the wheels free, plus fixed coloured blocks in front of the camera. Validates the serial protocol, servo end stops, motor direction, watchdog behaviour, threshold ranges and ultrasonic noise levels, with no risk of a crash.
2. **Component tests against recorded video.** The vision pipeline run against pre-recorded track footage and still frames, so that a threshold change can be evaluated repeatedly against identical input. This is how frame processing time, contour area thresholds and line-detection lockouts were verified off-track.
3. **Full-track runs.** The complete system on a competition-spec field, scored the way a judge would score it- laps completed, wall contacts counted by an observer, times taken on a stopwatch.

8.2 Gain tuning

The gains have no closed-form derivation (§6.2), so they were tuned by the standard procedure: set the derivative term to zero and raise the proportional gain until the vehicle tracks lane centre but visibly oscillates; add derivative until the oscillation damps without the response becoming sluggish; reduce proportional slightly if the vehicle overshoots on corner exit. The record of what each value did:

Phase	Gains tested	Observed behaviour
1 — high proportional	KP = 0.05, KD = 0	Severe S-curve oscillation at 245 PWM; the vehicle oversteered into the side walls on straights.
2 — derivative added	KP = 0.03, KD = 0.02	High-frequency oscillation damped out, but a mild weave remained near corner exits.
3 — final wall centring	KP = 0.015, KD = 0.01	Optimal. Smooth wall centring on straights at 245 PWM with no visible hunting. These are the committed values.
4 — pillar avoidance (R2)	cKP = 0.25, cKD = 0.25, cy = 0.08	Crisp, immediate pillar deflection with proximity scaling; no early lazy swerves at distance.

The wall-centring gains are an order of magnitude smaller than the pillar-avoidance gains because their inputs are on completely different scales: contour-area difference is a large integer in the thousands, while pillar offset is a pixel offset in the hundreds.

8.3 Results

All figures below are measured. Where a run failed, the cause is given rather than the run being discarded.

Metric	Target	Result	Detail
Open Challenge completion rate	—	9 / 10 (90 %)	Ten benchmark runs. The single failure was a floor-line reflection under direct halogen lighting, before the colour bounds were finalised.
Open Challenge 3-lap time	—	22.4 s mean	Mean over the 9 successful runs; ≈ 7.5 s per lap at 245 PWM on straights, 230 in corners.
Wall contacts per run	0	0.2	Two minor side-wall grazes across 10 full runs, both recovered by PD centring without stopping.
Obstacle Challenge sign obedience	100 %	34 / 36 (94.4 %)	Across 12 test runs, 3 pillars per run.
Signs displaced per run	0	0.08	One pillar contact across 12 runs, caused by an extreme close-range entry angle.

Metric	Target	Result	Detail
Parking success rate	—	13 / 15 (86.7 %)	Head-in parking benchmark, full footprint containment required to count as a success.
Vision loop rate	≥ 30 fps	50 fps	Measured with <code>time.perf_counter()</code> around frame capture and contour extraction; processing takes under 12 ms of the 20 ms budget.
Recalibration time at a new venue	< 15 min	~15 min	Timed at the previous round under LED venue lighting.

8.4 Test log

Date	Test target	Observation	Change made
10 July 2026	Baseline motor and serial link	Serial watchdog fired at 500 ms whenever the Pi paused, stopping the vehicle mid-run.	Added a 100 ms rate-limited heartbeat stream in <code>wro_serial.py</code> so the link stays alive between command changes.
14 July 2026	Colour segmentation, red vs orange	Red pillars were false-triggered by orange floor lines under warm lighting.	Split the ROIs by height (pillars $Y \leq 245$, floor $Y \geq 260$) and added an aspect-ratio filter, $H/W \geq 0.75$.
17 July 2026	Open Challenge turn exit	A fixed 2.0 s turn timer walked the vehicle into the outer wall when it entered a corner slightly fast.	Replaced the timer with vision-dynamic turn exit hold the turn until the opposite wall is re-acquired above contour area 600.
10 Aug 2026	Corner anti-drift tuning	Tyres drifted on sharp turns taken at the full 245 PWM.	Added dynamic speed reduction: drop to 195–230 PWM whenever steering deflection exceeds 30° .
12 Aug 2026	Direction locking	An oblique glimpse of the opposite-colour marker disrupted the turn direction and corrupted the lap count.	Made the first-colour track lock permanent for the run instead of re-evaluated per line.
13 Aug 2026	Return to home	The vehicle overshot the start line by about 10 cm through inertia after the third lap.	Reduced return speed to 175 PWM and added the 0.08 s active reverse brake pulse.
23 Aug 2026	Round 2 parking	The vehicle missed the magenta tape on our custom test track.	Adapted the parking engine to detect two red parking blocks and centre on their midpoint X. Note: this must be reverted to magenta detection for the official field- see §6.6 and Appendix C.

8.5 System-level measurements

Measurements taken across the whole vehicle, used to sanity-check the design decisions in §7.

Metric	Measured value	Significance
Vision loop rate	50 Hz (20 ms)	Roughly five times faster reaction than a sequenced ultrasonic array could deliver; costs about 1.8 W of Pi CPU power.
Nominal pack current	2.1 A at 245 PWM	Gives ~35 min of continuous running on a 2200 mAh 2S pack.
Peak surge current	3.8 A	Handled by the 5 A buck without a measurable output dip, thanks to the bulk capacitor.
Total vehicle mass	1240 g	Low enough that the 1.01 N of tractive force still gives 0.67 m/s^2 of acceleration.
Serial link latency	< 1.8 ms	Negligible against the 20 ms vision loop; the link is not the bottleneck.

9. Risk Register and Failure Modes

Risks marked **observed** are ones that actually happened to us. Risks marked **predicted** have not occurred but are credible. Separating the two matters: a register containing only predictions is a thought experiment, whereas one that records real failures and what they cost is an engineering record.

#	Risk	Likelihood	Impact	Mitigation	Status
R1	Raspberry Pi brownout under peak load	Was high	Run lost - the Pi reboots, it does not degrade	Moved to 2S LiPo with a dedicated 5 V / 5 A buck for the Pi and a 2200 μ F bulk capacitor on the 5 V bus (§5.2)	Observed twice, then closed. Zero recurrences in 100+ runs.
R2	Venue lighting shifts colour thresholds	High	Detection failure, run lost	Calibration procedure (§5.6) run during official testing time; Lab space chosen for walls specifically to reduce lighting sensitivity	Observed at the previous round. Mitigated by a rehearsed 15-minute procedure.
R3	Parking detector still tuned to red test blocks rather than official magenta markers	High if unchecked	Parking fails or triggers on the wrong object - large score loss	Magenta thresholds exist in masks.py; the detector path must be confirmed on an official field before the round (§6.6)	Open- first item on the pre-competition checklist
R4	Camera is the single point of perception failure	Low	Total loss of navigation	Failsafe stop on total wall loss (§6.4); ultrasonics fitted and available as a wall-following fallback but not yet used for it	Accepted, with a hedge
R5	Camera ribbon fatigue from vibration	Was medium	Intermittent dropped frames that look like software bugs	Ribbon enclosed in spiral wrap with strain relief at both ends; defensive null-frame guard in the vision loop	Observed, then closed
R6	Steering servo gear failure on impact	Was medium	Vehicle undriveable; run and practice session lost	Metal-gear MG996R with a spring-loaded servo saver arm to absorb impact energy	Observed once, then closed
R7	Camera mast vibration blurring frames	Was medium	Noisy contour areas, intermittent detection loss	Diagonal support strut to the front chassis plate; camera mounted on TPU isolation grommets; frame jitter down roughly 80 %	Observed, then closed
R8	Serial link drops mid-run	Low	Vehicle continues on its last command into a wall	500 ms firmware watchdog stops the motor and centres steering if the heartbeat ceases (§6.3)	Predicted. Watchdog implemented and bench-verified.
R9	Wrong plug into a USB-A port, applying drive voltage to a real USB device	Low	Hardware damage	Silkscreen labels; pre-round connector check as a fixed item on the setup checklist	Predicted. Weakly mitigated- labels only.
R10	Contact with the interior wall displacing it	Medium	Zero score for the round (Rule 9.18)	Failsafe stop; reduced cornering speed; measured contact rate of 0.2 per run (§8.3)	Predicted, partially observed as grazes
R11	Pack voltage sag late in a practice session shifting behaviour	Medium	Inconsistent results and misleading tuning	Tune only above 3.70 V per cell (7.40 V pack) under load; swap and balance-charge packs after every four practice runs	Managed by procedure

#	Risk	Likelihood	Impact	Mitigation	Status
R12	Start button not assigned in firmware; round launched from a terminal	Medium	Slow, error-prone start; a mistyped command at the wrong moment	Documented launch commands (§10.6); planned GPIO round-select toggle	Open, fix identified

10. Build and Reproduce

10.1 Bill of materials

Item	Qty	Notes
Raspberry Pi 5, 8 GB	1	Vision compute. The 8 GB variant is not strictly required at 640 × 480, but headroom made profiling easier.
Wide-angle camera module	1	Chosen over the original narrow-FOV module so both walls stay in frame through a corner (\$5.4)
ESP32-WROOM-32 development board	1	FCC ID 2AHMR-ESP32-S. Mounts on the RoboGuard carrier.
RoboGuard carrier PCB	1	Custom board hosting the ESP32, battery cradle, switch, LEDs and keyed peripheral ports
BO gearmotor	1	Rear drive, nameplate 300 rpm, 0.35 kg·cm. Exact gear ratio to be confirmed — see Appendix C.
MG996R steering servo	1	Metal geared, ≈ 12 kg·cm, fitted with a spring-loaded servo saver arm
H-bridge driver	1	Integrated on the RoboGuard board, not a separate module
Ultrasonic rangefinder	3	Front, left, right. Used as distance gates only (\$5.3).
Wheels with rubber tyres	4	68 mm diameter measured with tyre fitted; bought as a matched set
2S LiPo pack, 2200 mAh	1	7.4 V nominal. Brand and C-rating to be recorded — see Appendix C.
5 V / 5 A synchronous buck converter	1	Dedicated supply to the Pi 5 USB-C input
2200 µF low-ESR electrolytic capacitor	1	Bulk capacitance across the 5 V bus
Chassis and upper deck	2	Acrylic, 3.6 mm, CNC laser cut
Steering knuckles, tie rod, mounts, mast, sensor holders	set	PETG, 3D printed, 5–10 mm section
M3 screws, nuts, standoffs	set	---

Fabrication files. Mechanical parts were designed in Fusion 360; the exported files are linked from the repository. The RoboGuard board was produced from a stock reference design rather than an original schematic, so there is no in-house Gerber set to publish — this is stated plainly in Appendix C rather than left as an unexplained empty folder.

10.2 Mechanical assembly

1. Cut the chassis and upper deck from 3.6 mm acrylic and print the steering components, mast and sensor holders in PETG using the files referenced above.
2. Assemble the front axle: fit the king pins to the deck, fit the steering knuckles, and connect the tie rod. Check that the wheels swing freely through their full travel with no binding *before* the servo is fitted.
3. Fit the steering servo and link it to the tie rod. Centre the servo in software at 100° first, *then* attach the horn so the wheels point straight ahead doing it in the other order guarantees an asymmetric steering range.
4. Fit the rear axle as a single solid shaft and mount the BO gearmotor.
5. Fit the standoffs, mount the RoboGuard board with the battery cradle, then mount the Pi 5 on the upper deck.
6. Fit the camera mast with its diagonal brace, mount the camera on its TPU grommets, and route the camera cable in spiral wrap with slack at both ends so it cannot tug the camera out of alignment.
7. Fit the three ultrasonic holders — front, left, right — and dress their cables away from the motor leads.

10.3 ESP32 firmware

The firmware lives in the repository under `src/esp32_firmware/`:

File	Contents
<code>esp32_firmware.ino</code>	Packet parser, servo and motor drive, 500 ms safety watchdog
<code>platformio.ini</code>	PlatformIO project configuration
<code>README.md</code>	Pin definitions, packet format reference and flashing notes

Toolchain: Arduino IDE 2.x, or PlatformIO CLI on Espressif32 platform 6.4.0. **Board target:** `ESP32 Dev Module` (ESP32-WROOM-32, `esp32dev`).

Flashing procedure

1. Connect the ESP32 to the computer by USB.
2. Open `esp32_firmware.ino` in the Arduino IDE.
3. Select board `ESP32 Dev Module` and the active COM / tty port.
4. Ensure the libraries `ESP32Servo` (1.1.1 or later) and `ArduinoJson` (6.21.3 or later) are installed.
5. Upload — or run `pio run --target upload` under PlatformIO. Flashing completes in about 15 seconds.

10.4 Raspberry Pi software

```
sudo apt-get update
sudo apt-get install -y python3-opencv python3-numpy python3-pip
pip3 install pyserial
```

Confirm the ESP32 enumerates and that the user can open it:

```
ls -l /dev/ttyUSB*      # or /dev/ttyACM*, depending on the USB bridge
sudo usermod -aG dialout $USER
# log out and back in for the group change to take effect
```

10.5 Running

```
git clone https://github.com/mabamano/WRO_2026-RoboVanguard.git
cd WRO_2026-RoboVanguard

python3 raspberry_pi/open_challenge_R1.py      # Open Challenge
python3 raspberry_pi/obstacle_challenge_R2.py  # Obstacle Challenge

python3 raspberry_pi/test_motor_serial.py      # manual drive, no vision
python3 raspberry_pi/obstacle_challenge_R2_test_parking.py  # parking only
```

Press `q` in the HUD window to abort; the vehicle stops and the serial port closes cleanly.

10.6 Selecting and starting a round

How it works today. The round is selected by running the corresponding script from a terminal over SSH or on a directly attached monitor, before the vehicle is placed in the starting box. There is no physical round selector.

Why this is worth fixing. It is honest to say that a competition round should begin with a button press, not a keyboard. Launching over SSH means the operator is typing a command under time pressure, next to a field, with a laptop and typing the wrong script is a plausible mistake at exactly the wrong moment. The planned fix is a two-position toggle switch on a Pi GPIO read at boot by a systemd service, which selects the round script without any terminal at all. This is listed in §11.

11. Known Limitations and Roadmap

Stating these plainly is part of the engineering record.

#	Limitation	Planned fix	Priority
1	The parking detector is currently adapted to red test blocks rather than the official magenta markers	Confirm and commit the magenta detection path; verify on an official field during testing time	Critical
2	Steering is parallel at full lock rather than true Ackermann; the ideal outer angle is about 15° below the inner (§4.2)	Lengthen the steering arms and re-measure inner and outer angles at the stops	High
3	Motor shaft speed has never been measured; §4.4 rests on a nameplate figure the lap times do not support	Time the vehicle over a marked 2 m straight and correct the drive-train section	High
4	No physical round selector- the round is chosen by typing a command over SSH	Read a two-position toggle from a Pi GPIO at boot and launch the matching script from a systemd unit	High
5	The start button is present on the RoboGuard board but not assigned in firmware	Assign a GPIO, debounce it, and gate the run start on it	High
6	Parking stages 2 to 4 are open-loop and depend on timings tuned against one surface	Close the loop on left/right ultrasonic clearance during insertion	Medium
7	No independent navigation fallback if vision fails; ultrasonics are gates only	Add a coarse ultrasonic wall-following mode as a degraded-operation fallback	Medium
8	Solid rear axle scrubs in corners; roughly 35 % speed mismatch across the rear track at full lock	Fit a differential, or a one-way bearing on the inner rear wheel	Medium
9	The pin table in schemes/README.md predates this revision and contradicts the firmware	Regenerate it from the firmware pin definitions	Medium
10	The RoboGuard board is a stock reference design; there is no in-house schematic to publish	Redraw the board in KiCad for the next revision so the design is genuinely ours and publishable	Low

12. Repository Structure and Versioning

WRO_2026-RoboVanguard/	
├── README.md	Project overview
├── raspberry_pi/	Vision and control software
│ ├── open_challenge_R1.py	Open Challenge
│ ├── obstacle_challenge_R2.py	Obstacle Challenge
│ ├── obstacle_challenge_R2_test_parking.py	Parking harness
│ ├── wro_functions.py	Vision primitives
│ ├── masks.py	Colour thresholds
│ ├── wro_serial.py	Serial transport and heartbeat
│ └── test_motor_serial.py	Manual drive harness
├── src/esp32_firmware/	ESP32 firmware, PlatformIO project
├── schemes/	Wiring diagrams and board photographs
├── models/	Fabrication files for cut and printed parts
├── v-photos/	Vehicle photographs – six views
├── t-photos/	Team photographs
├── video/	Demonstration video links
└── other/	Supplementary documentation

12.1 Development trajectory

The codebase was built in four architectural phases rather than as one program, and the module boundaries reflect the order in which the problems were solved.

Phase	Modules created	What it established
1	wro_serial.py, test_motor_serial.py	Reliable 115200-baud packet streaming between the Pi 5 and the ESP32, with heartbeat and watchdog enforcement. Nothing else could be trusted until the link was.
2	masks.py, wro_functions.py	Lab and HSV thresholds, contour area extraction and layered wall segmentation, refined until colour classes stopped overlapping.
3	open_challenge_R1.py	PD wall centring, First-Colour Track Lock, vision-dynamic turn exit, lap counting and the return-to-home engine with active braking.
4	obstacle_challenge_R2.py, obstacle_challenge_R2_test_parking.py	PD pillar avoidance with vertical proximity scaling, emergency reversing, and the staged head-in parking manoeuvre.

12.2 Demonstration videos

The rules require **one video per challenge**, each showing at least 30 seconds of continuous autonomous driving.

Challenge	Status
Open Challenge	Video recorded — link in video/README.md
Obstacle Challenge	Video recorded — link in video/README.md

Appendix A — Vehicle Photographs

These photographs were taken before the revision 2.0 power rework and show the earlier single-cell battery arrangement. The chassis, steering, drive train and camera mast are unchanged.

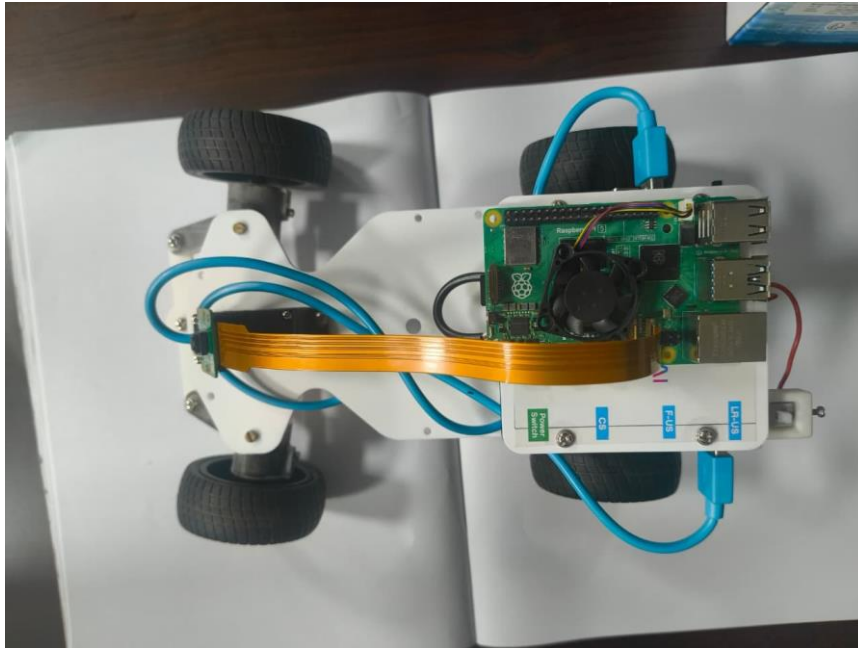


Figure A1 — Top view: Raspberry Pi 5 on the upper deck, RoboGuard board and battery below, camera cable routed forward.

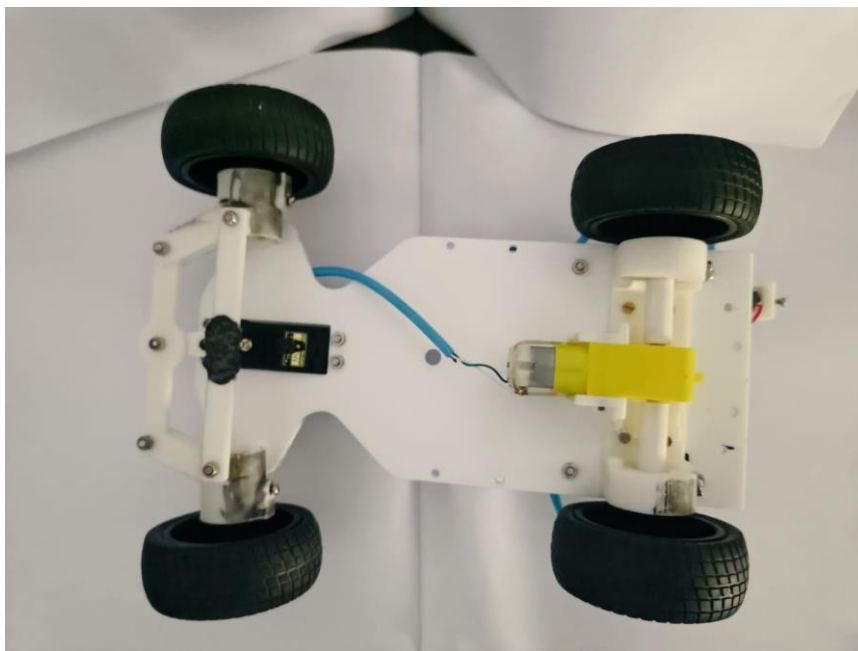


Figure A2 — Bottom view: chassis deck, rear BO gearmotor, steering servo and the Ackermann-type linkage.



Figure A3 — Front view: camera mast, steering knuckles and tie rod at full assembly.

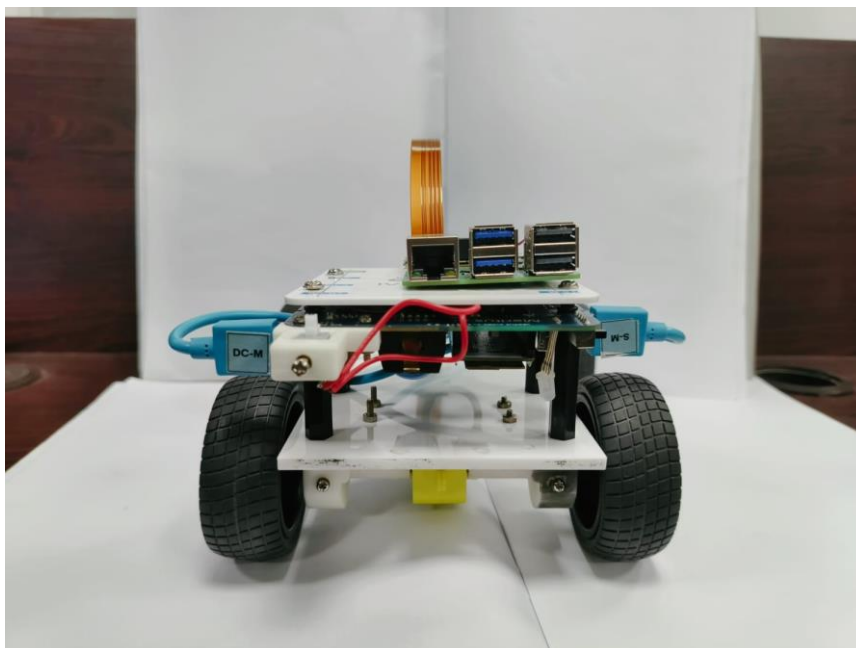


Figure A4 — Rear view: drive wheels, and the RoboGuard board with its labelled DC-M and M-S ports.

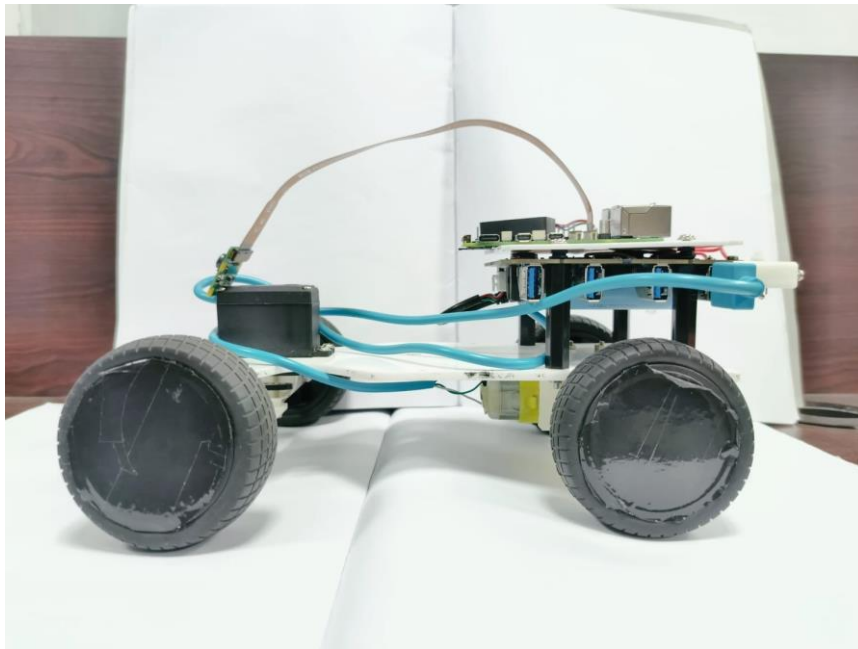


Figure A5 — Left side elevation.

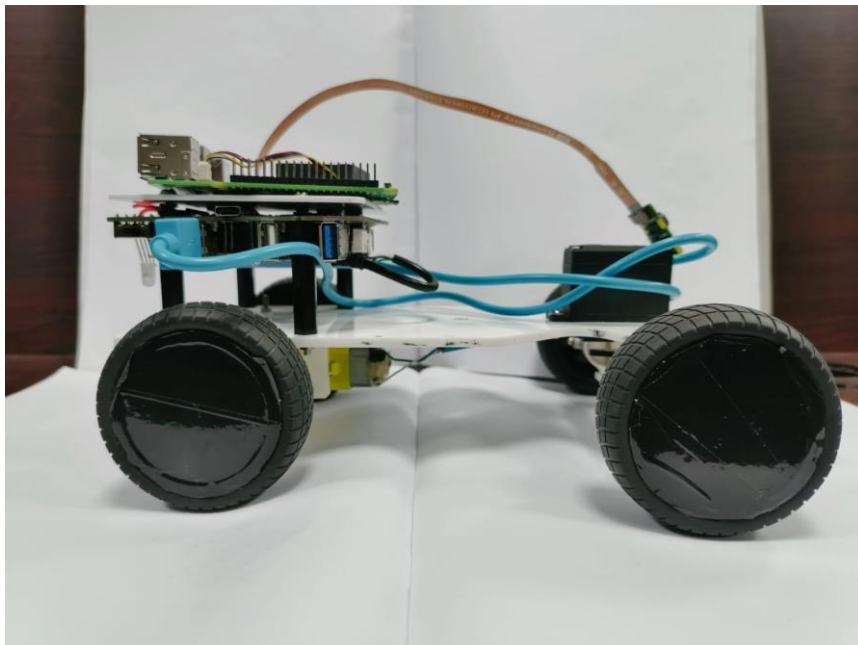


Figure A6 — Right side elevation, showing the two-deck stack and camera mast height.

Appendix B — Team



Figure B1 — Team ROBOVANGUARD, Ramco Institute of Technology.

Team ROBOVANGUARD · Ramco Institute of Technology · WRO 2026 Future Engineers